

LAB 1

Project Scaffold: App Router, TypeScript, and Your First Client Component

Course “Advanced Web Technologies” · Week 1 · 1 contact hour + independent completion before the next class

Goal

Scaffold the semester project — a course catalog built on Next.js 16 (App Router) — and put everything from Lecture 1 into practice: file-based routing, Server and Client Components, dynamic routes, and basic TypeScript typing. You'll keep extending this same project in every lab that follows.

Before you start

- Node.js version 20 or newer — check with: `node -v`
- a GitHub account and git installed
- a code editor (VS Code recommended)
- Lecture 1's handout open nearby — today you'll lean on its examples and Diagrams 2, 3, and 6

What you'll build

A small but fully working app with four routes:

```
route map
/           - the home page
/about      - a static page about the course
/courses    - the course list (data from the “server”)
/courses/[id] - a course page + a ❤️ button (Client Component)
```

The only data source for this lab is the mock functions in `lib/courses.ts`, simulating a backend (with a 300ms delay so `loading.tsx` is actually visible). A real backend arrives later, once we get to FastAPI.

1. Project Setup

Create the project with the command below. When the wizard asks: TypeScript — Yes, Tailwind CSS — Yes, ESLint — Yes, App Router — Yes, `src/` directory — No, import alias — leave it as `@/*`.

```
terminal
npx create-next-app@latest course-catalog
```

Move into the project folder and start the dev server — at this point the default Next.js starter page should open with no errors:

```
terminal
cd course-catalog
npm run dev
```

If the port is taken: Next.js will offer the next free port (3001, 3002...) — that's normal, just open whatever address it prints in the terminal.

Open `app/page.tsx` and `app/globals.css` and remove all the demo content left by `create-next-app` — we'll write our own next. Initialize a git repository and make your first commit:

```
terminal
git init
git add .
git commit -m "chore: bootstrap Next.js project"
```

2. Project Data

There's no real backend yet, so we'll simulate the data — exactly like `getCourses()/getCourse()` from the lecture. Create the file below exactly as it is — today it isn't the assignment itself, just a tool for the rest of the steps.

```
lib/courses.ts
export type Course = {
  id: string;
  title: string;
  description: string;
  credits: number;
  isElective: boolean;
  likes: number;
};

const courses: Course[] = [
  {
    id: "modern-frontend",
    title: "Modern Frontend: React & Next.js",
    description: "React 19, Server Components, and the App Router.",
    credits: 5,
    isElective: false,
    likes: 24,
  },
  {
    id: "backend-fastapi",
    title: "Backend Foundations: FastAPI",
    description: "Async REST APIs in Python with FastAPI and Pydantic.",
    credits: 5,
    isElective: false,
    likes: 19,
  },
  {
    id: "databases-postgresql",
    title: "Relational Databases: PostgreSQL",
    description: "Schema design, SQLAlchemy, and migrations with Alembic.",
    credits: 5,
    isElective: false,
    likes: 15,
  },
  {
    id: "api-design",
    title: "API Design: REST vs GraphQL",
    description: "Comparing REST and GraphQL in practice.",
    credits: 4,
    isElective: true,
    likes: 11,
  },
  {
    id: "web-security",
    title: "Web Security Essentials",
    description: "JWT/OAuth2, defending against XSS, CSRF, SQL injection.",
    credits: 4,
    isElective: false,
    likes: 21,
  },
  {
    id: "ai-integration",
    title: "AI/LLM Integration",
    description: "LLM features in an app, wired up via the OpenAI API.",
    credits: 5,
    isElective: true,
    likes: 32,
  },
];

function delay<T>(value: T, ms = 300): Promise<T> {
```

```

    return new Promise((resolve) => setTimeout(() => resolve(value), ms));
  }

export async function getCourses(): Promise<Course[]> {
  return delay(courses);
}

export async function getCourse(id: string): Promise<Course | undefined> {
  return delay(courses.find((c) => c.id === id));
}

```

3. Home Page and “About”

This is the simplest part — both pages are fully static, with no data.

app/page.tsx — route “/”

Server Component. Requirements:

- a heading (name your course catalog yourself) and a short welcome sentence;
- a link to /courses via the Link component from next/link — Link specifically, not a plain <a>, so navigation stays client-side with no full page reload.

app/about/page.tsx — route “/about”

A Server Component with any static text about the project: two or three sentences on what this catalog is and which course it was built for.

4. Course List

app/courses/page.tsx — route “/courses”

Server Component. Make sure to use await getCourses() directly in the component body — no useEffect, no loading state, exactly like section 1 of the lecture. Render the list of cards through the CourseCard component you'll write next.

components/CourseCard.tsx

Server Component. Prop contract — implement exactly this signature:

```

components/CourseCard.tsx — contract
type CourseCardProps = {
  id: string;
  title: string;
  description: string;
  credits: number;
  likes: number;
};

```

- the whole card is a `<Link href={` /courses/${id} `}>` with content inside (not `onClick`! navigation through Link doesn't require 'use client' — the component stays a server component);
- title, description, and credits are displayed inside;
- likes can simply be shown next to a heart — we'll make it interactive in section 6.

Hint: if the card won't type-check or complains about its props — check it against the CourseCard example in section 6.1 of the lecture notes, it walks through exactly this pattern.

5. Course Page: Dynamic Route

This is the most important part of the lab — this is where you'll apply params as a Promise, `generateStaticParams`, and the Server + Client Component pairing from sections 4 and 5 of the lecture.

`app/courses/[id]/page.tsx`

Server Component. Requirements for the signature and behavior:

- `params` is typed as `Promise<{ id: string }>` and must be awaited — if you forget the `await`, you'll get exactly the error section 5 of the handout warns about;
- course data comes from `await getCourse(id)`;
- if the course isn't found (`getCourse` returns `undefined`) — call the `notFound()` function from `next/navigation` instead of rendering an empty page;
- the page displays title, description, credits, and the `LikeButton` component (section 6).

`app/courses/[id]/loading.tsx`

A Client or Server Component (either works, there are no hooks here) with any simple loading text or indicator — for example, “Loading course...”. Thanks to the 300ms delay in `lib/courses.ts`, you'll actually see this screen on navigation.

`app/courses/not-found.tsx`

The component shown when `notFound()` is called — for example, “Course not found” with a link back to `/courses`. Easy to test: open `/courses/does-not-exist` in the browser.

`generateStaticParams`

In the same `app/courses/[id]/page.tsx`, add a function that tells Next.js to build the page for every course in the mock data ahead of time — the signature and purpose match the example in section 3 of the handout exactly:

```
app/courses/[id]/page.tsx - signature hint
export async function generateStaticParams() {
  // return an array like [{ id: "modern-frontend" }, { id: "backend-fastapi" }, ...]
  // using getCourses() from lib/courses
}
```

6. Interactivity: LikeButton

components/LikeButton.tsx

The only Client Component in the whole lab — don't forget 'use client' as the file's first line. Prop contract:

```
components/LikeButton.tsx - contract
type LikeButtonProps = {
  initialLikes: number;
};
```

- the like count is state — via `useState<number>(initialLikes)`;
- clicking the button increases the count by 1 (a real backend for likes arrives in a later lab — local state is enough for now);
- the button shows a heart and the current like count, e.g.: ❤️ 24.

Wire LikeButton into the course page from section 5, passing `initialLikes={course.likes}`.

7. Navigation and Styling

In `app/layout.tsx`, add simple navigation shared across every page — three links (Home / Courses / About) via `next/link`, styled with Tailwind classes (padding, spacing, a hover state — your call; design isn't graded on looks, only on working correctly).

That wraps up the functional part of the lab. What remains is the target folder structure, matching Diagram 2 from the handout:

```
final app/ structure
app/
├── layout.tsx           ← navigation
├── page.tsx             ← "/"
├── about/
│   └── page.tsx         ← "/about"
├── courses/
│   ├── page.tsx         ← "/courses"
│   ├── not-found.tsx
│   └── [id]/
│       ├── page.tsx     ← "/courses/:id"
│       └── loading.tsx
```

Acceptance Checklist

Before you submit, run through this list — it's exactly what will be checked:

- ☐ `npm run dev` starts up with zero TypeScript errors in the terminal;
- ☐ `npm run build` completes successfully (this also verifies `generateStaticParams`);
- ☐ all four routes open and show the expected content;

- ☐ `/courses/does-not-exist` shows the `not-found.tsx` page, not a blank screen or a crash;
- ☐ likes on the course page increase on click and don't reset before a page reload;
- ☐ `components/LikeButton.tsx` is the only file with `'use client'` in the whole project;
- ☐ the repository is pushed to GitHub with at least 3–4 meaningful commits (not one big “final version”).

Grading Rubric

Criterion	Points
Project initialized via <code>create-next-app</code> with TypeScript and Tailwind CSS	1
All four routes work: <code>/</code> , <code>/about</code> , <code>/courses</code> , <code>/courses/[id]</code>	2
<code>/courses/[id]</code> — correct params handling (<code>await</code>) and <code>generateStaticParams</code>	2
<code>not-found.tsx</code> and <code>loading.tsx</code> are wired up and actually trigger	1
<code>CourseCard</code> — a Server Component with typed props, navigation via <code>Link</code>	1
<code>LikeButton</code> — a Client Component with working <code>useState</code> , the only <code>'use client'</code> in the project	2
GitHub repository with a history of meaningful commits	1

Total: 10 points. Adjust the scale to your own grading system as needed.

Common Mistakes

A missing `await`: `params` needs to be awaited both in the App Router and in `generateStaticParams`. The error “`params should be awaited`” means a missing `await`, not a Next.js bug.

`'use client'` everywhere: if TypeScript complains about `useState` in `CourseCard` or `page.tsx` — that's a sign the interactive logic ended up in the wrong component. `useState` should only live in `LikeButton.tsx`.

`onClick` on a `Link`: don't try to handle a card click with `onClick` — that immediately requires `'use client'`. A plain `href` on `Link` is all the navigation needs.

`npm run dev` passes but `npm run build` doesn't: the dev server doesn't always catch every static-build error. Always run `npm run build` at least once before submitting.

Bonus Tasks (optional)

If you finish the core part early — pick any, no limit on how many:

- add `app/courses/error.tsx` and deliberately trigger an error (e.g., if `id === "broken"`) — check that the Error Boundary catches it instead of crashing the whole app;
- add client-side search by course title on `/courses`: the page itself stays a Server Component and passes the already-loaded list into a separate Client Component wrapper with a search field and local filtering;
- set up a route group (`auth/`) with a `/login` stub, matching Diagram 2 from the handout — no real authentication, just to practice the pattern;

4. deploy the project to Vercel (free tier) and add the live link to your repository's description.

How to Submit

Your GitHub repository should be named course-catalog and be public, or shared with the instructor.

In README.md, briefly describe what you implemented, and include a link to the deployment if you have one.

Send the repository link however your instructor specifies (LMS / course chat).

Deadline: before Lecture 2 begins, unless your instructor states otherwise.